Universidad Simón Bolívar
Departamento de Computación y Tecnología de la Información
CI-3661 — Laboratorio de Lenguajes de Programación I
Septiembre – Diciembre 2026

Prof. Leonardo López Almazán
leonardolopez@usb.ve

Tarea 1
Práctica de Haskell

15 puntos

Fechas

Publicación:	viernes 16 de octubre de 2026 (semana 3), después de la clase 3
Entrega:	**jueves 22 de octubre de 2026, 23:59** (semana 4)
Entrega tardía:	hasta el domingo 25 de octubre de 2026, 23:59, con descuento
Modalidad:	**individual**

1. Objetivo

Practicar lo visto en las clases 2 y 3 (viernes 9 y 16 de octubre): definir funciones por recursión y *pattern matching*, usar guardias, tuplas, `Maybe` y `Either`, restricciones de clases de tipos (`Eq`, `Ord`), y luego resolver problemas con funciones de orden superior (`map`, `filter`, `foldr`, `foldl`), listas por comprensión y lambdas.

La **Parte A** usa lo de la clase 2 y la **Parte B**, lo de la clase 3.

2. Reglas

- Trabaja sobre la plantilla `Tarea1.hs`. No cambies el nombre del módulo, la lista de exportación ni las firmas. Puedes agregar todas las funciones auxiliares que quieras, cada una con su firma.
- El único `import` permitido es `Data.Char`.
- **Parte A**: solo recursión explícita. Están prohibidos `map`, `filter`, `foldr`, `foldl`, las listas por comprensión, `reverse`, `maximum`, `minimum`, `all`, `any` y cualquier otra función de orden superior de la biblioteca.
- **Parte B**: lo contrario. Ninguna función de la Parte B, ni sus auxiliares, puede ser recursiva.
- El archivo debe cargar en GHCi sin errores. Se compila con `ghc -Wall`: las advertencias restan.
- Una solución que funciona pero viola las restricciones de su parte vale la mitad.

3. Parte A: recursión, patrones, `Maybe` y `Either` (7.5 pts)

Problema 1. Palíndromos (2 pts)

Escribe `esPalindromo :: String -> Bool`, que dice si una cadena se lee igual de izquierda a derecha que de derecha a izquierda, **ignorando** todo lo que no sea letra o dígito (`isAlphaNum`) y sin distinguir mayúsculas de minúsculas (`toLower`). No hace falta tratar los acentos: `á` y `a` son caracteres distintos. La cadena vacía es palíndromo.

```
ghci> esPalindromo "Anita lava la tina"
True
ghci> esPalindromo "1 2 3 2 1"
True
ghci> esPalindromo "Haskell"
False
```

Recuerda que no puedes usar `reverse`: si la necesitas, escríbela.

Problema 2. Mayor y menor en un solo recorrido (2 pts)

Escribe `mayorYMenor :: Ord a => [a] -> Maybe (a, a)`, que devuelve el par (mayor, menor) de una lista, o `Nothing` si está vacía. Debe recorrer la lista **una sola vez** (calcular el máximo y luego el mínimo por separado son dos recorridos). En el comentario que trae la plantilla, explica en dos o tres líneas por qué tu solución hace un solo recorrido.

```
ghci> mayorYMenor [3, 9, -2, 5]
Just (9,-2)
ghci> mayorYMenor ["mami", "homura", "sayaka"]
Just ("sayaka","homura")
ghci> mayorYMenor ([] :: [Int])
Nothing
```

Problema 3. Horas y duraciones (3.5 pts)

(a) Escribe `leerHora :: String -> Either String (Int, Int)`, que interpreta una hora de 24 horas en formato `HH:MM` y devuelve el par (hora, minutos). La cadena debe tener exactamente cinco caracteres: dos dígitos, dos puntos y dos dígitos. Los errores son, *con estos textos exactos*:

Situación	Resultado
No tiene la forma `dd:dd`	`Left "formato inválido"`
La hora es mayor que 23	`Left "hora fuera de rango"`
Los minutos son mayores que 59	`Left "minutos fuera de rango"`

Si la hora y los minutos están ambos fuera de rango, se reporta la hora. Pista: un patrón de lista puede tener literales, como `[a, b, ':', c, d]`. (1.75 pts)

(b) Escribe `duracion :: String -> String -> Either String Int`, que da los minutos que pasan desde la primera hora hasta la segunda. Si la segunda es anterior, se entiende que es del día siguiente. Si alguna hora es inválida, el resultado es el error de la *primera* hora inválida. Usa `leerHora` y `case`; todavía no hemos visto otra forma de encadenar cálculos que pueden fallar. (1.75 pts)

```
ghci> leerHora "07:45"
Right (7,45)
ghci> leerHora "24:10"
Left "hora fuera de rango"
ghci> duracion "08:30" "10:05"
Right 95
ghci> duracion "23:50" "00:20"
Right 30
ghci> duracion "08:30" "1005"
```

```
Left "formato inválido"
```

4. Parte B: orden superior (7.5 pts)

Problema 4. Promedio sobre un umbral, en un plegado (1.5 pts)

Escribe `promedioSobreUmbral :: Double -> [Double] -> Maybe Double`, que promedia los elementos *estrictamente* mayores que el umbral, o da `Nothing` si no hay ninguno. Debe hacerse con **un solo** `foldr` (o `foldl`) cuyo acumulador sea una tupla; no vale `filter` seguido de `sum` y `length`.

```
ghci> promedioSobreUmbral 2 [1, 3, 5, 2]
Just 4.0
ghci> promedioSobreUmbral 10 [1, 2, 3]
Nothing
```

Problema 5. Saltos de caballo (2 pts)

Las casillas de un tablero de ajedrez se representan como pares (fila, columna), con ambas coordenadas entre 1 y 8.

(a) Escribe, con una lista por comprensión, `movimientosCaballo :: (Int, Int) -> [(Int, Int)]`, que da las casillas a las que puede saltar un caballo desde una casilla dada, sin salirse del tablero y **ordenadas** lexicográficamente (primero por fila, luego por columna). No puedes ordenar después: tu comprensión debe generarlas ya en orden. Pista: piensa en qué desplazamientos hay y en qué orden los recorres. (1 pts)

(b) Escribe `enDosSaltos :: (Int, Int) -> Int`, el número de casillas *distintas* en las que puede quedar el caballo después de exactamente dos saltos (la casilla de partida cuenta, porque se puede ir y volver). Tendrás que eliminar repetidos sin recursión: un plegado sirve. (1 pts)

```
ghci> movimientosCaballo (1, 1)
[(2,3),(3,2)]
ghci> enDosSaltos (1, 1)
10
```

Problema 6. Todo es un `foldr` (1.5 pts)

Escribe `miMap` y `miFilter` usando solo `foldr` (0.75 pts), y `miFoldl :: (b -> a -> b) -> b -> [a] -> b`, que se comporta como `foldl`, usando solo `foldr` (0.75 pts). Para `miFoldl`, la idea es que el plegado no construya el resultado directamente sino una *función* que, al recibir el valor inicial, lo calcula.

```
ghci> miFoldl (-) 10 [1, 2, 3]
4
ghci> take 3 (miMap (+ 1) [1 ..])
[2,3,4]
```

Problema 7. Evaluación por sustitución, con tu carnet (2.5 pts)

Sea d la lista con los **cuatro últimos dígitos de tu carnet**, en orden. Por ejemplo, para el carnet `19-10437`, d = `[0, 4, 3, 7]`. Escribe a mano o en texto plano, paso a paso como en la clase 3, la

reducción de:

(a) `foldr (-) 0 d` hasta llegar a un número. (0.5 pts)

(b) `foldl (-) 0 d` hasta llegar a un número. (0.5 pts)

(c) Con **tu** definición de `miFoldl`, la reducción de `miFoldl (-) 0 [`d_1`, `d_2`]`, donde d_1 y d_2 son los dos primeros dígitos de d. Cada paso debe corresponder a una ecuación de tu código o a una regla de β-reducción. (0.75 pts)

(d) Explica en un párrafo por qué (a) y (b) dan distinto, y da una condición sobre el operador `op` y el valor `z` que garantice que `foldr op z xs == foldl op z xs` para toda lista finita `xs`. (0.75 pts)

Una traza con un carnet que no es el tuyo vale cero.

5. Uso de asistentes de IA

Puedes usar asistentes de IA para preguntar conceptos, entender un mensaje de error o revisar ideas, igual que usarías un libro o le preguntarías a un compañero. Lo que no puedes es entregar como tuyo código que no escribiste. Al inicio de `Tarea1.hs` hay un espacio para declarar qué herramientas usaste y para qué; si no usaste ninguna, escribe "ninguna". Declarar el uso no resta; no declararlo y que se note, sí.

6. Material

- `Tarea1.hs`: plantilla con las firmas.
- `PruebasTarea1.hs`: pruebas públicas. Para correrlas, ponlo en la misma carpeta que tu `Tarea1.hs` y ejecuta: `runghc PruebasTarea1.hs`

Que pasen las pruebas públicas no garantiza la nota: hay pruebas privadas con más casos borde, y se revisan las restricciones de cada parte y el estilo.

7. Entrega

- Crea un repositorio **público** en GitHub llamado `ci3661-t1-<carnet>` (por ejemplo `ci3661-t1-19-10437`) y envía el enlace a `leonardolopez@usb.ve`, con asunto `[CI3661] Tarea 1 - <carnet>`, apenas lo crees.
- Trabaja en el repositorio desde el principio, con commits que vayan mostrando tu avance. No importa cuándo trabajes, pero un único commit con toda la tarea se penaliza.
- Pon la etiqueta `entrega` (`git tag entrega` y `git push -tags`) en la versión que quieres que se califique.
- En la raíz del repositorio deben estar:
  - `Tarea1.hs`, con tu nombre, carnet y la declaración de uso de IA.
  - `traza.txt` o `traza.pdf` con el problema 7 (un PDF puede ser una foto legible de una hoja escrita a mano).
  - `README.md` con tu nombre y carnet.

Repositorios públicos

Los repositorios son públicos, así que cualquiera puede verlos. Copiar de otro repositorio es plagio, y el historial de commits muestra quién escribió cada cosa primero.

Entregas tardías

Se aceptan entregas hasta el **domingo 25 de octubre, 23:59**, con un descuento de 3 puntos por cada día de retraso o fracción. Después no se recibe la tarea. Se califica lo que haya en GitHub a la hora límite; si haces cambios después del jueves, cuentan como entrega tardía.